Task 1: Temperature

In [70]:
```
import requests
import pandas as pd
import matplotlib.pyplot as plt
from frost_id import client_id

client_id = client_id

#Requests the daily mean air temperture for all of 2025
endpoint = 'https://frost.met.no/observations/v0.jsonld'
parameters = {
'sources': 'SN17850',
'elements': 'mean(air_temperature P1D)',
'referencetime': '2025-01-01/2025-12-31',
}

# Issue an HTTP GET request
r = requests.get(endpoint, parameters, auth=(client_id,''))
# Extract JSON data
data = r.json()
```

Make a nicely formatted plot of this air temperature, where time is on the x-axis and temperature is on the y-axis

In [83]:
```
observations = data["data"]

df = pd.DataFrame(observations)


df["temperature"] = df["observations"].apply(
    lambda x: x[0]["value"]
)

df["time"] = pd.to_datetime(df["referenceTime"])

df.head()
```

Out[83]:

	sourceId	referenceTime	observations	temperature	time
0	SN17850:0	2025-01-01T00:00:00.000Z	[{'elementId': 'mean(air_temperature P1D)', 'v...	-5.2	2025-01-01 00:00:00+00:00
1	SN17850:0	2025-01-02T00:00:00.000Z	[{'elementId': 'mean(air_temperature P1D)', 'v...	-11.5	2025-01-02 00:00:00+00:00
2	SN17850:0	2025-01-03T00:00:00.000Z	[{'elementId': 'mean(air_temperature P1D)', 'v...	-8.9	2025-01-03 00:00:00+00:00
3	SN17850:0	2025-01-04T00:00:00.000Z	[{'elementId': 'mean(air_temperature P1D)', 'v...	-15.5	2025-01-04 00:00:00+00:00
4	SN17850:0	2025-01-05T00:00:00.000Z	[{'elementId': 'mean(air_temperature P1D)', 'v...	-12.5	2025-01-05 00:00:00+00:00

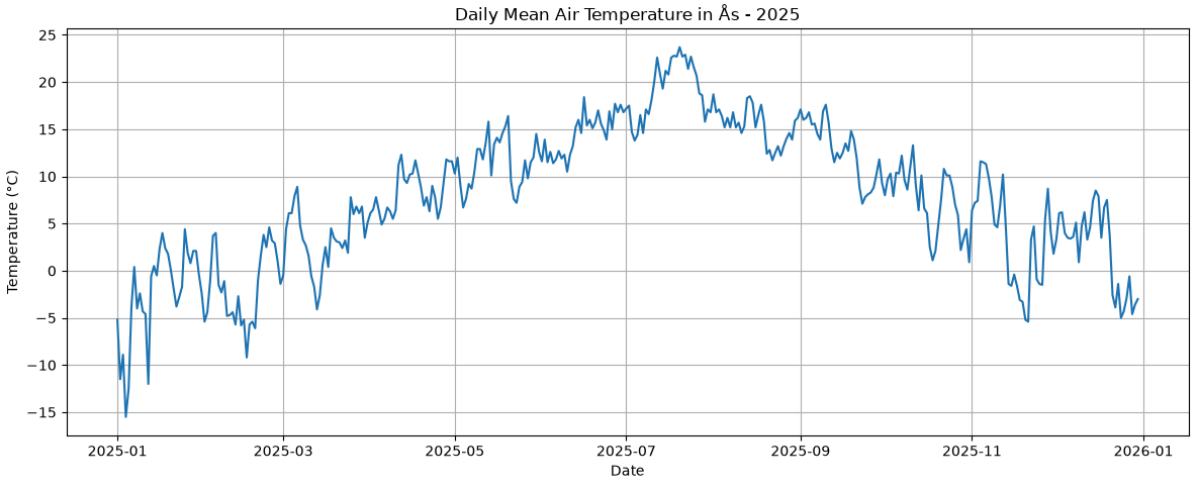
In [72]:

```
plt.figure(figsize=(12, 5))

plt.plot(df["time"], df["temperature"])

plt.xlabel("Date")
plt.ylabel("Temperature (°C)")
plt.title("Daily Mean Air Temperature in Ås - 2025")

plt.grid(True)
plt.tight_layout()
plt.show()
```



Create a nicely formatted summary table containing mean, median, min and max temperature for this year, and display it with print()

In [74]:

```
summary_table = pd.DataFrame({
    "Statistic": ["Mean", "Median", "Min", "Max"],
    "Temperature (°C)": [
        df["temperature"].mean(),
        df["temperature"].median(),
        df["temperature"].min(),
        df["temperature"].max()
    ]
})

print(summary_table)
```

```
  Statistic  Temperature (°C)
0      Mean          7.902747
1    Median          8.400000
2       Min        -15.500000
3       Max         23.700000
```

Discussion: Task 1

The plot shows the daily mean air temperature in Ås during 2025. We can see how the temperature changes throughout the year by season.

The summary table presents the mean, median, minimum, and maximum daily temperatures. The mean describes the average temperature over the year, while the median represents the middle value of the observations. The minimum and maximum show the coldest and warmest daily mean temperatures recorded in the dataset.

Task 2: Combined graph

In [75]:
```
#Requests the daily mean air temperture for all of 2025
endpoint = 'https://frost.met.no/observations/v0.jsonld'
parameters = {
'sources': 'SN17850',
'elements': 'sum(precipitation_amount P1D)',
'referencetime': '2025-01-01/2025-12-31',
}

# Issue an HTTP GET request
r = requests.get(endpoint, params=parameters, auth=(client_id,''))


# Extract JSON data for precipitation
precipitation_json = r.json()

#Precipitation to a Pandas dataframe
precipitation_data = pd.DataFrame(precipitation_json["data"])

# Get the data from where it is placed in the data (in observations)
precipitation_data["precipitation"] =
precipitation_data["observations"].apply(
    lambda x: x[0]["value"])

#Time converted to Datetime with pandas
precipitation_data["time"] =
pd.to_datetime(precipitation_data["referenceTime"])


# Combining data before plotting. Refrences the data from the task 1
Combined_data = pd.merge(df[["time","temperature"]],
precipitation_data[["time","precipitation"]], on="time")

print(Combined_data)
```

```
                       time  temperature  precipitation
0   2025-01-01 00:00:00+00:00         -5.2            5.4
1   2025-01-02 00:00:00+00:00        -11.5            0.0
2   2025-01-03 00:00:00+00:00         -8.9            0.1
3   2025-01-04 00:00:00+00:00        -15.5            0.0
4   2025-01-05 00:00:00+00:00        -12.5            0.5
..                        ...          ...            ...
359 2025-12-26 00:00:00+00:00         -2.9            0.1
360 2025-12-27 00:00:00+00:00         -0.6            0.0
361 2025-12-28 00:00:00+00:00         -4.6            0.0
362 2025-12-29 00:00:00+00:00         -3.6            0.0
363 2025-12-30 00:00:00+00:00         -3.0            0.0

[364 rows x 3 columns]
```

In [76]:
```
#Plotting the data

# Creates a figure
fig, ax1 = plt.subplots(figsize=(14, 6))

#Plots temperature on the first axis
ax1.plot(Combined_data["time"],
         Combined_data["temperature"],
         color="red")

#Names the axises
ax1.set_xlabel("Date")
ax1.set_ylabel("Tempertaur (°C)", color="red")

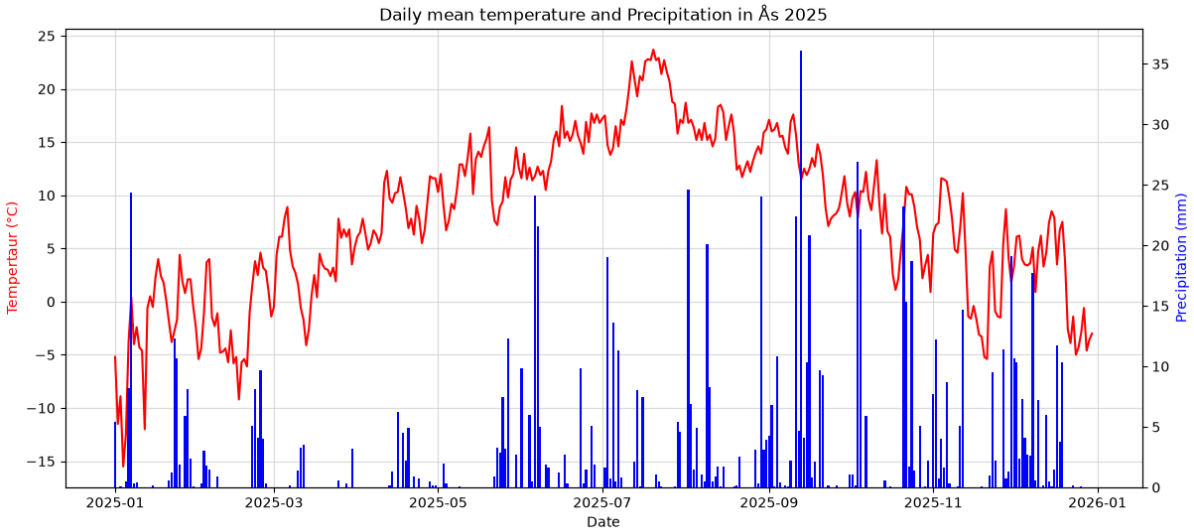
# Creates a second axis for precipitation since it and temperature dont have
the same scale
ax2 = ax1.twinx()

ax2.bar(Combined_data["time"],
        Combined_data["precipitation"],
        color="blue")

ax2.set_ylabel("Precipitation (mm)", color="blue")

# Adds a grid based on ax1
ax1.grid(True, alpha=0.4)
#Title on the graph
plt.title("Daily mean temperature and Precipitation in Ås 2025")

plt.show()
```



Discussion: Task 2

The end result of this task you can see above. The graph represent the tempreature through the year and the amount of precipitation through that same period. This code was sort of a part 2 for task 1, so certain parts of the info used were defined in that task. The easiest way to solve it ended up being by seperating all the specific info needed, checking that it was correct, and then

merging it back togheter before plotting it. The colours and style-choice used in the graph was chosen simply because they were the most readable ones. The colour blue is usually associeted with water and rain, and the same can be said for red and temperature. You can clearly see that the highest temperatures was measured during the summer, and the coldest during the winter. The rain also made itself known during the summer, but the biggest downpours happend during the fall

Task 3: Historical temperature data

Now you want to compare the 2025 temperatures to temperatures measured 100 years earlier.

Get these temperatures from the database.

Plot the new and the old temperatures in the same plot, still nicely formatted

Generate a summary table as for task 1, where you display data from both the new

and the old temperatures.

Plot

In [78]:
```
# Requests the daily mean air temperature for all of 1925
endpoint = 'https://frost.met.no/observations/v0.jsonld'

parameters_1925 = {
    'sources': 'SN17850',
    'elements': 'mean(air_temperature P1D)',
    'referencetime': '1925-01-01/1925-12-31',
}

# Issue an HTTP GET request
r_1925 = requests.get(
    endpoint,
    parameters_1925,
    auth=(client_id, '')
)

# Extract JSON data
data_1925 = r_1925.json()

observations_1925 = data_1925["data"]

df_1925 = pd.DataFrame(observations_1925)

df_1925["temperature"] = df_1925["observations"].apply(
    lambda x: x[0]["value"]
)

df_1925["time"] = pd.to_datetime(df_1925["referenceTime"])

df["day"] = df["time"].dt.dayofyear
df_1925["day"] = df_1925["time"].dt.dayofyear
```

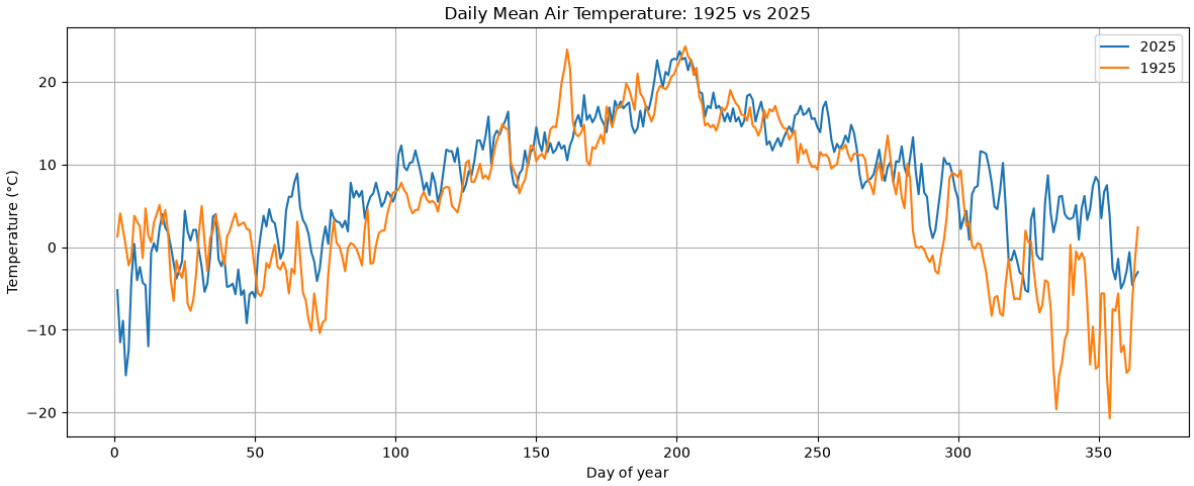
```
# Create a new figure for the comparison plot
plt.figure(figsize=(12, 5))

plt.plot(
    df["day"],
    df["temperature"],
    label="2025"
)

plt.plot(
    df_1925["day"],
    df_1925["temperature"],
    label="1925"
)

plt.xlabel("Day of year")
plt.ylabel("Temperature (°C)")
plt.title("Daily Mean Air Temperature: 1925 vs 2025")

plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()
```



In [79]:
```
#Tabell
summary_table = pd.DataFrame({
    "Statistic": ["Mean", "Median", "Min", "Max"],
    "1925 (°C)": [
        df_1925["temperature"].mean(),
        df_1925["temperature"].median(),
        df_1925["temperature"].min(),
        df_1925["temperature"].max()
    ],
    "2025 (°C)": [
        df["temperature"].mean(),
        df["temperature"].median(),
        df["temperature"].min(),
        df["temperature"].max()
    ]
})

print(summary_table)
```

```
  Statistic  1925 (°C)  2025 (°C)
0      Mean    5.31044   7.902747
1    Median    5.30000   8.400000
2       Min  -20.70000 -15.500000
3       Max   24.30000  23.700000
```

Discussion: Task 3

The plot compares daily mean air temperatures in Ås in 1925 and 2025. Both years are plotted against the day of the year, allowing comparison of temperatures from similar times of year.

The summary table compares the mean, median, minimum and maximum temperatures for both years. These statistics help identify differences in the overall temperature distributions.

Task 4: Heatwaves

In [89]:
```
import yaml

client_id = client_id

#Requests the daily mean air temperture for all of 2025
endpoint = 'https://frost.met.no/observations/v0.jsonld'
parameters = {
'sources': 'SN17850',
'elements': 'max(air_temperature P1D)',
'referencetime': '2025-01-01/2025-12-31',
}

# Issue an HTTP GET request
r = requests.get(endpoint, parameters, auth=(client_id,''))
# Extract JSON data
data = r.json()
```

```
#Function that takes a station_id and year as input and returns a list of
dates of when it was a heatwave
def heatwaves(station_id, year):
    hot_days = []
    print(f"Amount of observations in data: {len(data['data'])}")
    for observation in data['data']:
        if station_id in observation['sourceId'] and str(year) in
observation['referenceTime']:
            if observation['observations'][0]['value'] ⩾ 27:
                hot_days.append(observation['referenceTime'])

    #Checks if the days are after each other
    def find_heatwaves(hot_days):
        if not hot_days:
            return []

        # Turns into dates and sort and removes any dates that might be the
same
        dates = pd.to_datetime(hot_days).sort_values().unique()

        #Empty lists
        heatwave_periods = []
        current_streak = []

        for dato in dates:
            #If the list is empty or the date is the day after the last
            if not current_streak or (pd.Timestamp(dato) -
pd.Timestamp(current_streak[-1])).days == 1:
                current_streak.append(dato)
            else:
                # If the row ended it checks if it was 5 days long or longer
                if len(current_streak) ⩾ 5:
                    start_date = current_streak[0].strftime("%Y-%m-%d")
                    end_date = current_streak[-1].strftime("%Y-%m-%d")
                    heatwave_periods.append(f"Fra {start_date} til
{end_date}")

                # Starts a new row with this date
                current_streak = [dato]

        # Checks the last row when the loop ends
        if len(current_streak) ⩾ 5:
            start_date = current_streak[0].strftime("%Y-%m-%d")
            end_date = current_streak[-1].strftime("%Y-%m-%d")
            heatwave_periods.append(f"Fra {start_date} til {end_date}")

        return heatwave_periods
    result = find_heatwaves(hot_days)
    return result
```

```
heatwave_days_2025 = heatwaves('SN17850', 2025)
print(heatwave_days_2025)


#Creates a summary of the heatwaves in the form of a yaml file

#Creates a dictionary first (is more practical prior to crateing the file)
Summary = {
    "title": f"Summary of heatwaves in 2025",
    "station_id": "SN17850",
    "year": 2025,
    "heatwave_dates": heatwave_days_2025  # List of dates
}

#Creates the yaml file and writes the summary in it
with open('heatwave_report.yaml', 'w') as file:
    yaml.dump(Summary, file, default_flow_style=False)
```

```
Amount of observations in data: 364
['Fra 2025-07-16 til 2025-07-20']
```

Discussion: Task 4

The purpose of this task was to check for heatwaves in 2025. I chose to solve it by splitting the task into 2: first filtering out the days where the temperature was high enough, and then cheking wheater or not they were consecutive. When that was all done the infor was put into a short summary, and said summary was transformed into a yaml-file. The end result is that we had one heatwave during 2025

Task 5: Moving average temperature

Write a function that calculates the 7-day moving average for temperature

Use this function on the 2025 mean temperature you collected in task 1

Make a plot showing the moving average as a line and the daily temperature as points

In [82]:
```
def moving_average(data : pd.Series, window_size : int = 7) -> pd.Series:
    """a function that calculates the 7-day moving average for temperature
data"""

    return data.rolling(window=window_size).mean()

df["time"] = pd.to_datetime(df["referenceTime"])

df["moving_average"] = moving_average(df["temperature"])

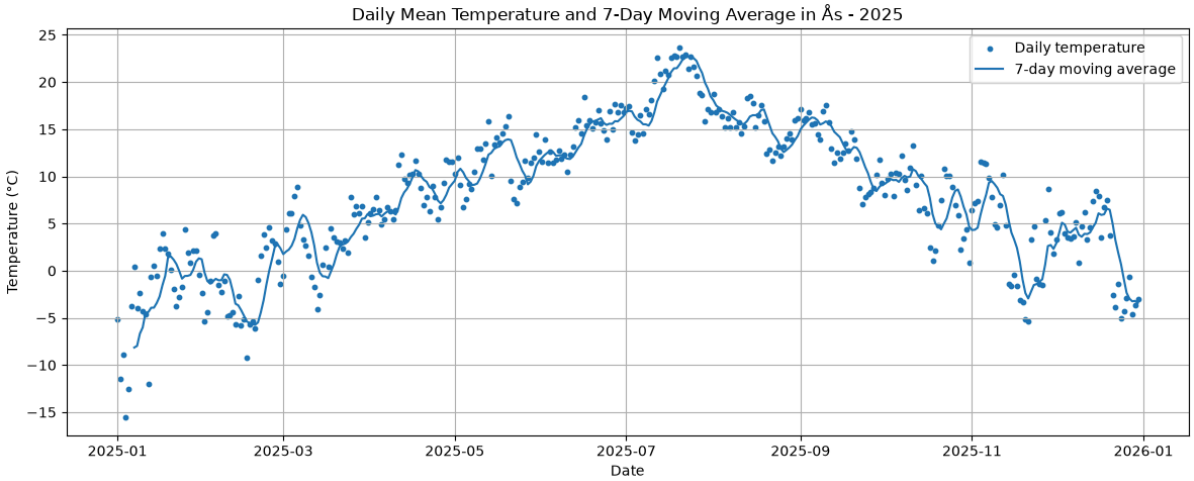

plt.figure(figsize=(12, 5))

plt.scatter(
    df["time"],
    df["temperature"],
    label="Daily temperature",
    s=10
)

plt.plot(
    df["time"],
    df["moving_average"],
    label="7-day moving average"
)

plt.xlabel("Date")
plt.ylabel("Temperature (°C)")
plt.title("Daily Mean Temperature and 7-Day Moving Average in Ås - 2025")

plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()
```



Discussion: Task 5

The plot shows the daily mean temperature and the 7-day moving average in Ås during 2025. Daily temperatures are displayed as points, while the moving average is shown as a line.

A moving average allows us to interpret temperature trends in a more generalized way. Unlike daily observations, it reflects the average temperature over a period of seven consecutive days. Since a moving average requires at least 7 data points, the first six values are unavailable, which is a standard characteristic of this type of calculation.

Such an average complements daily temperature data and makes it easier to identify broader shifts in temperature patterns.